



Politehnica University of Timisoara



Web Programming

C7. Emerging Web Programming Technologies



Emergent technologies

- Web programming platform continues to evolve rapidly
- Objectives:
 - richer client applications / augmented reality
 - faster server responses / efficient distribution
 - new language choices / improved security
- Expanding the execution model (binary code, GPU)
- Edge and serverless computing, WebIoT
- AI-driven Semantic Web / Web 4.0
- AI-assisted development



Augmented reality

- Immersive experiences within the browser
- Examples
 - Retail websites can provide virtual try-ons, allowing buyers to see how garments or accessories would look
 - Real estate can provide virtual property tours, allowing customers to walk around rooms and get a sense of the space as if they were present in person





Web AR applications

- A combination of web technologies and a device's built-in hardware
 - Camera and Sensors - the user's device camera captures the real-world environment, then sensors like accelerometer, gyroscope, and GPS track the position, orientation, and location
 - Data Processing (e.g., using WebAR SDK) - computer vision algorithms analyze the camera input to understand the scene

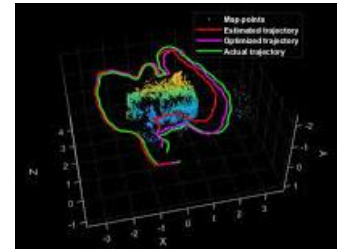


Ref: <https://docs.blippar.com/webar-sdk/>



Web AR applications

- AR-related tasks - recognizing surfaces using the SLAM technique (Simultaneous Localization and Mapping), or identifying specific images in the scene, e.g., using marker-based augmented reality
- Rendering - using WebGL or WebXR, 3D models and other digital content are overlaid onto the live camera view in real time
- Content Delivery - virtual assets and data are loaded from a database or a WebCMS, allowing real-time content updates without updating the application



Ref: <https://www.mathworks.com/discovery/slam.html>



WebAR tools and technologies

- Platforms/WebAR SDKs - Niantic 8th Wall, Zappar, Vuforia, or Adobe Aero    
- New Web Technologies - dedicated JavaScript libraries such as AR.js, A-Frame, and Three.js are commonly used to render 3D graphics and manage interactions
- Future Standards - WebXR is an emerging standard API that aims to provide a unified framework for both **AR** and **VR** experiences on the web, for powerful and immersive browser-based AR   

* WebXR samples: <https://immersive-web.github.io/webxr-samples/>



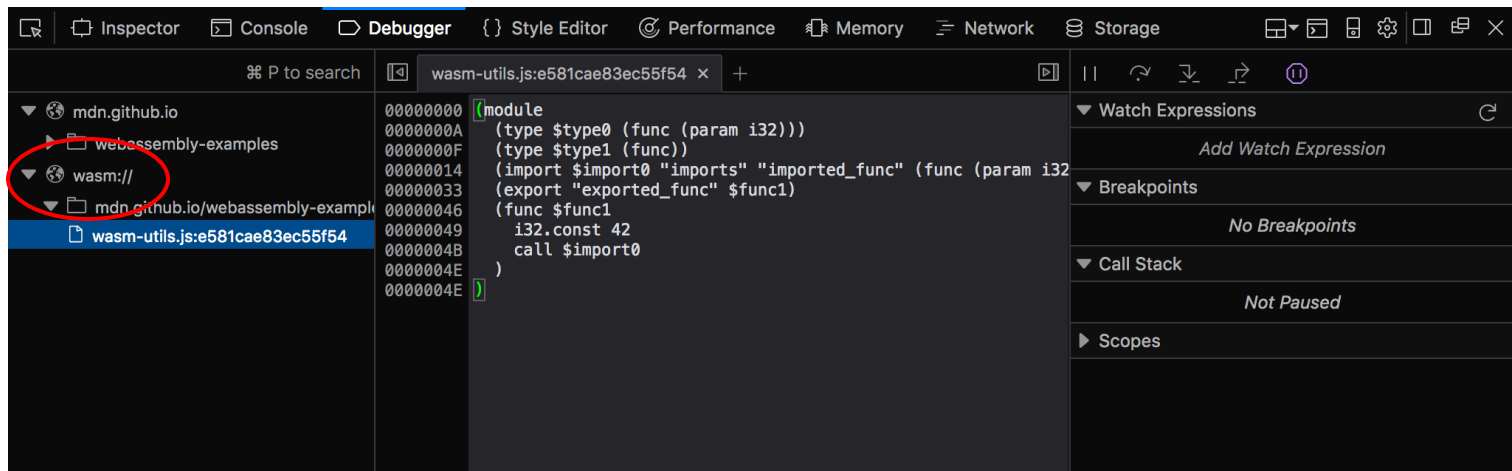
WebAssembly (Wasm)

- A compact, portable **binary instruction format** that provides near-native performance across browsers and non-browser environments
- It is the compilation target for languages such as Rust, C/C++, etc., allowing CPU-heavy code (multimedia, crypto, complex algorithms) to run efficiently
- It includes WASI (WebAssembly System Interface) - interfaces for file, network, and time operations to enable Wasm modules to run securely on servers, edge platforms, and embedded devices (not just in browsers)



Viewing Wasm in Firefox

- Firefox Developer Tool - Debugger Panel exposes the text representation of any Wasm code included in the web page. Use "wasm://" entry in the Panel



Ref: https://developer.mozilla.org/en-US/docs/WebAssembly/Guides/Using_the_JavaScript_API



WebGPU

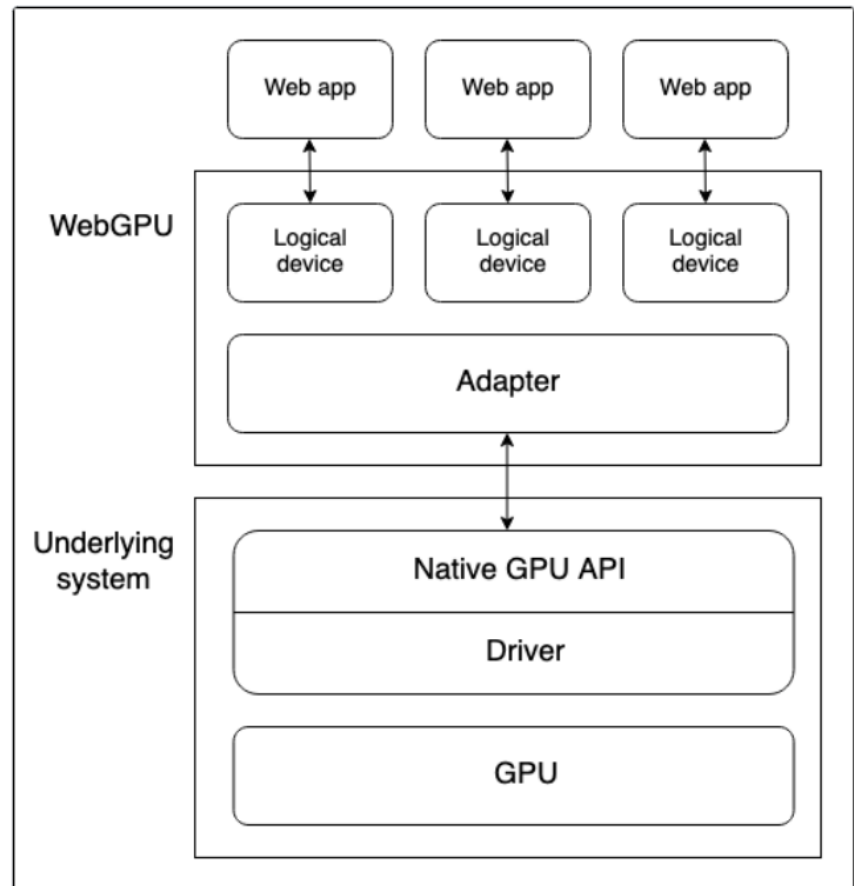
- Successor of WebGL, exposes general GPU computation and modern rendering features to web developers
- Empower web applications to do advanced rendering, ML inference, and general-purpose GPU (GPGPU) tasks directly in the browser
- Ensure lower overhead, compute shaders, and richer synchronization in Web apps
- Used for implementing heavy client-side rendering (e.g. 3D games), image processing, or on-device ML models evaluation





The WebGL model

- Can work with
 - Integrated GPUs on the same board as the CPU that share its memory
 - Discrete GPUs, on their own board, separate from the CPU
 - Software emulated "GPUs", implemented on the CPU



Ref: https://developer.mozilla.org/en-US/docs/Web/API/WebGPU_API



JS runtimes: Deno and Bun

- They do not replace Node.js overnight, but present solid alternatives for new projects and service components
- **DenoJS** provides secure defaults, built-in TypeScript support, and an integrated toolchain. It aims to reduce configuration and improve security
- **Bun** emphasizes raw performance and an all-in-one experience (bundler, package manager, test runner) built on JavaScript Core
- Both runtimes are popular and improve startup time, dependency management, and native tooling



Deno vs. Node.JS

Comparison Between NodeJS vs DenoJS



Features	NodeJS	DenoJS
Language	JavaScript	JavaScript & TypeScript
Packages	Npm modules	.ts or .js script
Security	Full Access	Permissioned Access
Code Engine	V8 JavaScript	V8 JavaScript
TypeScript Support	Not Built in	Built in
Community	Strong & Huge	Relatively New

Ref: <https://www.rlogical.com/blog/nodejs-vs-denojs-which-is-better-for-you/>



Edge and serverless

- Main goal: pushing compute resources closer to users
- Edge platforms (**Cloudflare Workers**, **Fastly**, etc.) run code on globally distributed points, reducing latency and enabling new architectures (personalized edge caching, request-time computation)
- These platforms increasingly support multiple languages and Wasm modules, providing low-latency code run outside centralized data centers
- They improved response times and new cost / performance tradeoffs for APIs and content delivery



Cloudflare Workers apps

- **Front-end applications** - deploy static assets to Cloudflare's CDN (Client Centric Approach) and cache them for fast rendering
- **Back-end applications** - build APIs and connect to data stores with Smart Placement to optimize latency
- **Serverless AI inference** - run LLMs, generate images, and more with Workers AI
- **Background jobs** - schedule Cron jobs, run durable Workflows, and integrate with Queues
- **Observability & monitoring** - monitor performance, debug issues, and analyze traffic with real-time logs and analytics





WebIoT

- Is a leading paradigm for creating interoperable, scalable, and user-friendly connected systems
- Uses standard web protocols and data formats to connect heterogeneous IoT devices
- Enable real-time IoT device control and monitoring through browsers and web services
- Extends IoT by applying web architecture principles — Uniform Resource Identifiers (URIs), HTTP, and RESTful interfaces — to make devices accessible as standard web resources



WebIoT Programming (I)

- RESTful APIs and HTTP Communication
 - REST (Representational State Transfer) is the backbone of WebIoT programming. Each device exposes resources (e.g., /temperature, /led) accessible via HTTP methods — GET, POST, PUT, DELETE — used by web clients, dashboards, or automation scripts, scalable due to stateless communication
- WebSockets for Real-Time Communication
 - For low-latency, bidirectional communication, WebSockets are preferred. Devices or gateways establish persistent connections to web servers, allowing instant updates — useful for sensor streaming or live dashboards (e.g., energy monitoring or smart home control)



WebIoT Programming (II)

- MQTT and CoAP for Constrained Environments
 - MQTT (Message Queuing Telemetry Transport) over WebSocket is a lightweight publish/subscribe protocol for data transfer designed for low-power or bandwidth-limited devices
 - CoAP (Constrained Application Protocol) is designed for constrained devices, mirrors HTTP, but operates over UDP, offering lower overhead in embedded environments
- Web Integration and Visualization
 - Front-end frameworks such as React or Vue are used to render IoT data dynamically. Developers often employ Web APIs like Fetch or Axios for data retrieval, and D3.js or Chart.js for real-time visualization



WebIoT Programming (III)

- Edge and Cloud Connectivity
 - WebIoT systems often combine edge computing (processing data near sensors) with cloud APIs for storage and analytics
 - Frameworks such as Node-RED (built on Node.js) allow rapid development of IoT workflows through graphical programming and REST/MQTT integration
- Security and Data Management
 - WebIoT security focuses on HTTPS/TLS encryption, token-based authentication (JWT, OAuth2), and CORS (Cross-Origin Resource Sharing) policies for safe browser-device communication
 - JSON and MQTT are used for structured message exchange



WebIoT Workflow Example

- Typical WebIoT workflow steps:
 - Sensor nodes (ESP32, Raspberry Pi) collect data
 - They send data via MQTT or HTTP POST to a gateway
 - The gateway publishes data to a cloud API or database
 - A web dashboard retrieves and visualizes the data in real time using WebSockets or REST calls
- Ref: [1] W3C, Web of Things (WoT) Architecture (2023), <https://www.w3.org/WoT/>
[2] OpenJS Fnd., Node-RED Documentation (2025), <https://nodered.org/docs/>



AI-driven Semantic Web

- Extends traditional web capabilities by enabling machines to understand, reason about, and act upon data meaningfully rather than processing it syntactically
- **Semantic Web or Web 3.0** uses RDF, OWL, and SPARQL to represent, link, and query data semantically
- Recent advances in AI – NLP and Knowledge Graphs – have accelerated the Semantic Web's capabilities
- Semantic Web becomes **Intelligent Web or Web 4.0** - systems can infer meaning, discover relationships, and make autonomous decisions



Core AI-Driven SW Techniques

- **Knowledge Graphs (KG)** - structured representations of entities and their relationships using RDF triples and ontologies, supporting reasoning and question answering learned from unstructured text
- **Ontology Learning and Reasoning** - deep learning and symbolic reasoning automate ontology creation, reducing manual effort
- **NLP and Semantic Search** - models like BERT, GPT, and Word2Vec empower semantic search and understanding by mapping natural language queries to semantic concepts. Coupled with RDF data, these models enable semantic question answering systems
- **Semantic Data Integration and Interoperability** - AI-driven alignment techniques match heterogeneous data schemas across domains. ML can automatically map similar classes and properties across ontologies, improving data interoperability



Key Features of Web 4.0

- **AI at the Core** - backbone of Web 4.0 enables Web apps to predict what users want before they even ask
- **NLP** - adds the ability to understand human language
- **IoT Integration** - connect the digital world with the physical world. Refrigerators, cars, and even clothing will share data and work together with Web apps
- **Decentralization with Blockchain** – reduces reliance on centralized platforms like Facebook or Google, allowing individuals to own their digital identities, ensuring privacy and security
- **AI-based AR and VR** - provide an immersive experience



Challenges of Web 4.0

- **Privacy Concerns** - personalized services require collecting large amounts of data, raising questions about how this information is stored and used
- **Cybersecurity Threats** - as systems become more interconnected, they also become more vulnerable to cyberattacks
- **Overdependence on Technology** - relying too heavily on intelligent systems reduces human decision-making skills
- **Complexity** - developing and maintaining Web 4.0 infrastructure will require heavy resources, which could limit adoption in certain regions



AI-assisted development

- Generative AI tools (**GitHub Copilot**, **ChatGPT**, etc.) now offer code and test generation for both client and server-side
- Public reports indicate substantial productivity gains in routine tasks, but raise concerns about security, license compliance, and the propagation of subtle bugs or insecure patterns in AI-generated code
- Organizations must enforce measures like code review policies, security scanning, and human verification for AI-produced source code





References

- WebAssembly official site — overview and specs <https://webassembly.org>
- WASI (WebAssembly System Interface) — specs <https://wasi.dev>
- MDN WebGPU specs (W3C) and API documentation https://developer.mozilla.org/en-US/docs/Web/API/WebGPU_API
- Deno official site and blog <https://deno.com/>
- Bun official site and runtime coverage <https://bun.com/>
- Cloudflare Workers documentation (edge/serverless patterns) <https://developers.cloudflare.com/workers>